

Breaking Bad: How Compilers Break Constant-Time Implementations

Moritz Schneider
ETH Zurich
Zurich, Switzerland
moritz.schneider@inf.ethz.ch

Daniele Lain
ETH Zurich
Zurich, Switzerland
daniele.lain@inf.ethz.ch

Ivan Puddu
ETH Zurich
Zurich, Switzerland
ivan.puddu@inf.ethz.ch

Nicolas Dutly
ETH Zurich
Zurich, Switzerland
nicolas.dutly@inf.ethz.ch

Srdjan Capkun
ETH Zurich
Zurich, Switzerland
srdjan.capkun@inf.ethz.ch

Abstract

The implementations of most hardened cryptographic libraries use defensive programming techniques for side-channel resistance. These techniques are usually specified as guidelines to developers on specific code patterns to use or avoid. Examples include performing arithmetic operations to choose between two variables instead of executing a secret-dependent branch. However, such techniques are only meaningful if they persist across compilation. In this paper, we investigate how optimizations used by modern compilers break the protections introduced by defensive programming techniques. Specifically, how compilers break high-level constant-time implementations used to mitigate timing side-channel attacks. We run a large-scale experiment to see if such compiler-induced issues manifest in state-of-the-art cryptographic libraries. We develop a tool that can profile virtually any architecture, and we use it to run trace-based dynamic analysis on 44,604 different targets. Particularly, we focus on the most widely deployed cryptographic libraries, which aim to provide side-channel resistance. We are able to evaluate whether their claims hold across various CPU architectures, including x86-64, x86-i386, armv7, aarch64, RISC-V, and MIPS-32.

Our large-scale study reveals that several compiler-induced secret-dependent operations occur within some of the most highly regarded hardened cryptographic libraries – even when the high-level source code was formally verified to be free of side channels. To the best of our knowledge, such findings represent the first time these issues have been systematically observed in the wild and provide concrete data confirming previous speculations about defensive programming techniques’ limitations. One of the key takeaways of this paper is that the state-of-the-art defensive programming techniques employed for side-channel resistance are still inadequate, incomplete, and bound to fail when paired with the optimizations that compilers continuously introduce.

CCS Concepts

• **Security and privacy** → **Systems security; Software and application security.**



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASIA CCS '25, Hanoi, Vietnam

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1410-8/25/08

<https://doi.org/10.1145/3708821.3733909>

Keywords

Constant time code, cryptographic implementations, compilers

ACM Reference Format:

Moritz Schneider, Daniele Lain, Ivan Puddu, Nicolas Dutly, and Srdjan Capkun. 2025. Breaking Bad: How Compilers Break Constant-Time Implementations. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*, August 25–29, 2025, Hanoi, Vietnam. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3708821.3733909>

1 Introduction

Since the discovery of timing attacks [24], side-channel vulnerabilities have been one of the major concerns for developers of security-critical code and libraries [21]. Particular attention and expert knowledge are devoted to avoiding side-channel issues in security-critical libraries. Three main hardening techniques approaches are generally followed: i) manual assembly hardening, ii) using special compilers that provide constant time guarantees, and iii) hardening the source code. However, all of these approaches suffer from practical limitations.

The first option is vetting hand-written assembly either by a developer [43] or with automated tools [10]. However, this limits code portability and requires manual effort for each supported (micro-)architecture—in practice, only the most popular architectures are analyzed in this manner (e.g., x86 [10]). However, security-critical libraries are expected to be deployed everywhere, leaving less vetted architectures as second-class citizens in terms of security and hence potentially more susceptible to attacks. A solution to this problem is to compile the portable source code of security-critical libraries with special compilers that automatically remove side channels [11, 38]. However, these compilers suffer from a set of shortcomings: support for processor architectures is poor, they might require expert knowledge (e.g., to annotate the code), and they struggle to provide support for modern features of the processor or the employed source code languages. As a consequence, this approach is rarely used in practice, e.g., the binaries of security-critical libraries provided by Linux packaging repositories are compiled with commodity compilers such as GCC and LLVM. The third and final approach relies on hardening the higher-level source code using defensive programming techniques such as constant-time programming and then compiling the hardened code using commodity compilers. The advantage of this approach is that commodity compilers, as opposed to special compilers, offer support for many architectures,

are maintained and get improvements by hundreds of developers, and support the latest CPU features. Some projects even go as far as formally verifying the hardened higher-level code [50]. However, commodity compilers might apply transformations or optimizations that re-introduce side-channel vulnerabilities—an issue that surveyed developers of security-critical libraries are aware and afraid of [21], and that has been previously observed in small manually crafted examples [13, 42].

As a practical compromise between the three techniques mentioned so far, modern security-critical libraries rely on defensive programming techniques on high-level source code and the occasional spot check in their resulting binaries [21]. While there exist multiple recommendations on how to write hardened high-level source code [4, 5], and these have been applied to various projects [14, 29, 37, 44], it remains unclear whether they are effective at preventing *compiler-induced* side-channels. Furthermore, most of these coding recommendations were proposed before 2010, and thus, they may no longer apply to modern compilers (and/or need updates for novel optimizations). Moreover, new processor architectures differ significantly, and compilers may leverage different architecture-specific optimization techniques. Hitherto, it has not been studied how robust these recommendations are w.r.t. modern compilers and build environments.

In this paper, we study arguably some of the most hardened projects: cryptographic libraries, as they need to handle long-term secrets. Cryptographic libraries are usually developed by experts with a particular sensibility towards security and defensive programming. Many such libraries consider side-channel attacks in their threat models and aim to be side-channel free [14, 18, 29, 37, 50], and their developers often diligently follow hardening practices. These implementations are often called constant-time to indicate that they behave exactly the same for different secret inputs. Some libraries even provide formal guarantees for the absence of side channels [50], while others use various tools and manual analysis to eliminate side channels [14].

To understand if such constant-time development practices survive compilation, we ran a large-scale experiment to see if compiler-induced side-channel vulnerabilities exist in constant-time implementations used in the wild. We further aimed to understand how often they appear, which kind, and the reasons thereof. To answer this, we developed a scalable pipeline that allows us to analyze different compiler options for any specified target algorithm. We used our approach to perform a systematic study of the final binary of eight widely used cryptographic libraries on six different processor architectures, compiled with several versions of two common off-the-shelf compilers (GCC and LLVM) and with different optimization levels. This results in 6608 distinct binaries. Each binary contains multiple cryptographic primitives (e.g., AES, ECDSA, etc.) resulting in the total of 44,604 targets to test (i.e., experiments)¹. In each experiment, we tested a different binary and a different cryptographic primitive for their side-channel susceptibility. We used trace-based dynamic analysis [19, 47] to detect secret-dependent

control flow and memory accesses, the two most prominent root causes of side-channel vulnerabilities.

We discover many novel compiler-induced secret-dependent operations in state-of-the-art cryptographic libraries. These findings show that previously considered safe code patterns are being transformed into secret-dependent operations at the binary level. Notable findings include issues in libraries that were formally verified to be free of such side channels [50]. We discovered similar issues in all studied processor architectures, all compilers, and all optimization levels, including the recommended defaults. Our data indicates mainstream processor architectures, such as x86-64 and aarch64, to be less affected. We believe this is due to the developer’s focus on such architectures.

Our findings demonstrate that currently applied defensive programming techniques are not always effective at mitigating side-channel attacks, are very fragile, and may be vulnerable to new releases of compilers. Where possible, we investigate the compiler implementation to pinpoint what is causing the secret dependencies to be re-introduced in otherwise constant-time-looking source code. Generally, we found that compilers understand certain arithmetic patterns and replace them with secret-dependent branches or memory accesses. Given these findings, our work highlights the need for a more comprehensive and better foundation for defensive programming techniques against side channels.

Contributions.

- We study how compilers break constant-time properties of source code and, therefore, introduce timing side-channels across major cryptographic libraries and architectures.
- We perform a large-scale study of 8 cryptographic libraries and 11 cryptographic algorithms on 6 different processor architectures, compiled by 2 off-the-shelf compilers with 9 and 13 respective compiler versions under 7 different optimization levels, leading to a total of 44,604 experiments.
- We demonstrate compiler-induced side channels in state-of-the-art hardened cryptographic libraries and find several such issues in multiple libraries. For LLVM and for specific non-constant-time transformations, we pinpoint the compiler pass that causes these issues.
- This paper highlights that defensive programming techniques for side-channel resistance are ineffective, incomplete, and fragile with modern compilers.

Disclosure. We disclosed our findings to the respective developers of the libraries where we found any secret dependence (both compiler-induced and code-level issues): BearSSL, BoringSSL, Botan, HACL*, and Libsodium. Our disclosure included minimally reproducible examples of how certain compilers can break some parts of their implementation. The developers have acknowledged our findings, and two have provided fixes. The other library developers are currently working on a fix. The initial disclosure messages are provided in Appendix A.

2 Background

Side-channel Attacks. Side-channel attacks have been studied extensively in the past years. There are various different types of such attacks measuring, e.g., power consumption [23], changes in

¹Each binary contains multiple cryptographic primitives. For instance, one of the 6608 binaries is OpenSSL compiled with LLVM 8 using -O2 for x86-64. The compiled binary contains implementations for AES-GCM, HMAC-SHA1 etc. One experiment consists of testing AES-GCM from this binary. Another experiment consists of testing HMAC-SHA1 from the same binary, etc.

thermal emanations [32], cache usage [28], or the CPU branch predictor [1]. Physical side-channel attacks require the attacker to be in physical proximity of the device, e.g., to measure a power trace [23]. Digital side-channels can be executed remotely, e.g., by the attacker executing some other code on the same device [28]. Especially in the area of digital side channels, the attacker capabilities have been improved in the past few years with higher granularity [33, 49] and higher synchronization up to individual instructions [45]. In this paper, we focus on such advanced digital side channels.

Variations exploited by side channels can be measured across the whole execution or at a finer granularity, e.g., functions, branches, and even single instructions [45]. These variations might be induced by the internal state of the CPU or some other component that is accessible to the attacker executing on the same platform. Usual examples are the shared cache [28] and the shared branch-predictor [1, 27]. Unless the attacker can observe the leaky shared state directly, a common technique to exploit these variations is to convert them into timing variations.

Constant-time Programming. To counteract side-channel attacks, a crucial strategy in defensive programming is constant-time programming [4, 5]. This methodology involves adhering to a set of rules that seek to eliminate any secret-dependent branching operation, memory access, and sometimes even instructions that are known to produce timing differences based on their arguments. A well-known example is the select function depicted in two different implementations in Listing 1, which returns either x or y based on a secret bit. A non-constant-time-aware implementation of the select function (Listing 1a) would use an if statement to decide which value to return – thus branching depending on a secret value. Defensive programming proposes to replace the conditional statement with, e.g., the technique shown in Listing 1b, that achieves the same result by using bit arithmetic without secret-dependent branches. Common techniques include using binary arithmetic to replace branches, removing early exit conditions of loops, and avoiding table look-ups indexed by secret data (e.g., s-boxes). We note that these transformations can be automated [11, 38].

```
1 uint32_t select(uint32_t x, uint32_t y, uint32_t bit) {
2     if (bit)
3         return x;
4     else
5         return y;
6 }
```

(a) Not constant-time.¹²³

```
1 uint32_t ct_sel(uint32_t x, uint32_t y, uint32_t bit) {
2     // Create a bitmask in constant-time:
3     // - 0x0 if bit == 0
4     // - 0xFFFFFFFF if bit != 0
5     uint32_t mask = create_bitmask(bit);
6     // Select value to return in constant-time
7     return (x & mask) | (y & ~mask);
8 }
```

(b) Constant time.

Listing 1: An example select function from [5] that returns either x or y based on a secret bit. (a) is not constant-time due to branching on the secret value; (b) uses defensive programming techniques (bit arithmetic) to run in constant-time.

3 Problem Statement

Defensive programming techniques are often used to harden security-critical code to reduce the number of vulnerabilities and developer errors. Writing constant-time code is one such technique that seeks to mitigate side channels at the source code level [4, 5], and that experts rely on [37]. These techniques aim to remove any secret-dependent memory accesses and control-flow operations, thus eliminating secret-dependent variability and limiting the side-channel susceptibility of these libraries. The intuition is that if no secret-dependent operations exist, then no side channel can exist either.

Writing small assembly code by hand to achieve constant-time execution is common in cryptographic libraries. Usually, small critical routines are implemented in this fashion. However, this approach requires manual analysis for each architecture and thus needs a high level of effort and expertise for every supported architecture. Consequently, most projects use high-level source-code implementations with constant-time programming techniques. However, a compiler compiles high-level source code into a binary, which may optimize defensive programming techniques and reintroduce secret-dependent operations.

We aim to verify if such defensive programming techniques effectively mitigate side-channel attacks down to the binary level, considering today's compilers and available architectures, by analyzing arguably the most hardened libraries: cryptographic libraries. Compilers and their different versions, optimization levels, and target architectures could affect the final binary. Therefore, we aim to verify whether the resulting binaries have any secret-dependent operations – and whether the compiler introduced them.

Note that performing this analysis is challenging in practice because it requires being able to execute and profile binaries on many architectures in an automated fashion. Existing tools for analyzing such binaries are usually not portable and only work on one architecture, e.g., Intel Pin [20, 31, 47] for Intel Processors, or LLVM-based approaches that only allow analyzing LLVM output [19].

Scope. We focus our study on secret-dependent memory access and control flow operations that might be introduced by a compiler.² Since we focus on compiler-induced issues, we restrict the study to hardened side-channel free implementations. This makes it easy to isolate the impact of the compiler since if any issue is found in the binary, it must have been introduced by the compiler. Cryptography libraries are good candidates due to their threat model: generally, they consider any secret dependency issue to be critical, and if those are absent, then the library itself can be considered side-channel free. We do not consider implementations that use blinding to hide timing variations.

Not all secret-dependent operations are equal. Some might immediately leak the entire key, others only a single bit, and some might not be exploitable at all. We note that for such a study, the severity of the secret-dependent operations is irrelevant; they should not be present altogether if defensive programming techniques are followed. While the existence of secret-dependent operations does not imply that relevant information can be recovered, future attacks might improve this, e.g., by using a novel microarchitectural structure [40] or by increasing their granularity [2] – this is why

²We do not consider instructions that exhibit variable timing based on their arguments.

the prevalent defensive programming techniques for side-channel hardening aim at removing secret dependencies altogether.

4 Study Design

We developed an analysis pipeline to study compiler-induced side-channel issues, as depicted in Figure 1. We first designed a build system that allows us to target different versions of different compilers and to specify lists of possible values for different parameters, e.g., target architecture and optimization level. For each library we intended to study, we developed small applications that call the official APIs exposed by the libraries, e.g., `crypto_box` in Libsodium. Both the library and the small applications are then compiled using the selected parameters. Finally, the resulting binaries are analyzed using our tool, which finds secret-dependent operations and pinpoints their location in the analyzed library.

4.1 Detecting Secret-Dependent Operations

We developed a new tool [16] based on emulating user-space binaries through Qiling [26] and Unicorn [35], which are based on QEMU. This emulation-based approach allows targeting any processor architecture and compiler, as it can process any final binary. At the same time, our tool can detect the same issues as similar binary instrumentation frameworks [47]. To fix potential non-determinism, our tool hooks a set of syscalls selected by hand to fix the randomness to a deterministic value that does not change across multiple executions. Thus, by running in a fully deterministic environment, we posit that if we observe different execution traces for a given algorithm with two different secret keys, there is a secret dependency. Our tool emulates the execution of the target binary a configurable number of times and records the execution trace of each run. We manually disable certain JIT optimizations in QEMU that impact the traces we want to collect. Finally, our tool generates a report that states how many differences were found in traces, the functions that led to differences, and how often these were observed. Locating differences between multiple traces is trivial for memory-access traces but gets more involved for control flow. Just as in related work [46–48], we compare the control-flow traces until the first difference is detected. Afterward, we look for a merge point where the traces re-align. This approach is quite compute-intensive as it requires a forward search with multiple traces and comparisons. We note that no further differences will be flagged between the trace difference and its merge point. Therefore, this approach only provides a lower bound for control-flow issues.

In general, our approach is similar to other trace-based dynamic analysis tools [47, 48] that compare sets of memory address traces for different secrets. The main difference is in the use of emulation, which allows us to fix non-determinism and enables us to analyze many different architectures and compilers. We note that we merely refine these trace-based approaches and inherit all their shortcomings and advantages. Nevertheless, a study like the one presented in this paper is impossible with existing tools.

Soundness. To verify the correctness of our tool, we reproduced all the results found in DATA [47] for symmetric primitives in OpenSSL. In addition, we went through a significant number of flagged issues by hand to verify that they were real issues. We did not find any false positives.

Limitations. We note that by fixing the randomness, we also fix the nonce and thus do not detect nonce leakage (that could also lead to compromised keys) as we remove any potential nonce dependencies. Similarly, we do not detect any dependencies on the (fixed) plaintext or random numbers. Our approach could analyze all these one at a time, but we defer this to future work.

Our study uses a dynamic analysis approach. Thus, we can only find issues but not prove the absence thereof. We further arbitrarily set the number of runs to limit compute time. As a consequence, we might miss edge cases that only appear for some special conditions and would appear in a larger number of runs. Nevertheless, our findings demonstrate that we find relevant secret dependencies (c.f., Section 6), albeit we might have found even more with a higher number of runs. Our approach shows similarities with (and shares some limitations of) fuzzing systems that are also limited in their completeness and their execution time.

4.2 Study Setup

We conduct a large-scale analysis on two off-the-shelf compilers – 9 versions of GCC and 14 versions of LLVM – with 8 popular cryptographic libraries under 7 different optimization levels. This results in a total of 6608 distinct compiled binaries. We then analyze the binaries for up to 11 cryptographic primitives – depending on the library – resulting in 44,604 *experiments*, i.e., compiled high-level functions. We then collect 8 traces for each experiment.

We executed our study on eight machines equipped with an Intel Core i7-11700 and 32 GB of DRAM, and an AMD-based 192-core server with 1.5 TB of DRAM. Our experiments took around seven days of computing time.

Targeted Cryptographic Libraries. For this study, we selected 8 cryptographic libraries with varying claims of side-channel resistance: HACL* [50], Libsodium [14], Botan [29], BearSSL [37], BoringSSL [18], OpenSSL [43], WolfSSL [36], and MbedTLS [44]). We chose libraries that are widely used and/or highly regarded for their constant-time implementations. According to their documentation, all of these libraries strive to provide side-channel resistance at the source code level. HACL* [50] claims to achieve this fully. We note that many of the side-channel-related claims are usually not made with respect to the resulting binary on all architectures using any compiler. Instead, side-channel resistance claims are usually only targeted at a limited set of architectures and compilers [14] or for some intermediate representation [50]. We analyzed the library's latest release or stable version as of March 2023:

- HACL*/Project Everest [50] (commit: f283af1)
- Libsodium [14] (commit: 8cc84df)
- Botan [29] (commit: 15dc32f)
- BearSSL [37] (commit: 79c060e)
- BoringSSL [18] (commit: 28f96c2)
- OpenSSL [43] (commit: a92271e)
- WolfSSL [36] (commit: 4fbd4fd)
- MbedTLS [44] (commit: 1873d3b)

Cryptographic Algorithms. We also selected a set of algorithms to be investigated. The chosen algorithms should be supported by as many target libraries as possible and reflect a wide range

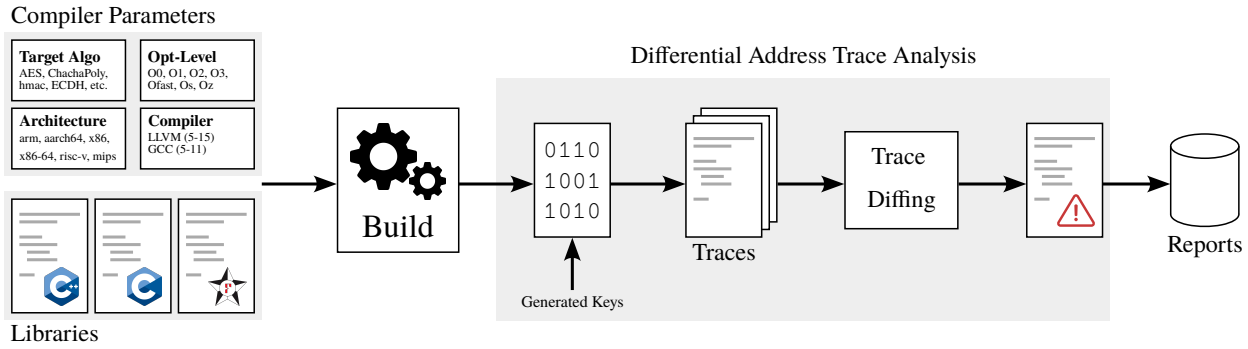


Figure 1: Pipeline used for our analysis. We use a dynamic analysis approach based on differential address trace analysis[19, 47].

of cryptographic primitives, from symmetric encryption and message authentication codes to cryptographic signatures and key exchanges. We list the chosen algorithms and their respective support for constant-time implementations in the libraries in Table 1.

Compilers and Parameters. We compiled the target libraries using one of 9 versions of GCC (from GCC 5 to GCC 13) or one of 14 versions of LLVM (from LLVM 5 to LLVM 18). We used MUSL as the underlying C standard library [17]. Note that we do not trace MUSL itself and thus will not detect any secret dependency within it. We do this because we are interested in studying the cryptographic libraries themselves and not a specific libc environment in which they might run. Since LLVM does not ship with a compatible sysroot, we fall back on the sysroot provided by the GCC toolchain.

We also investigated how optimization levels impact secret-dependent memory accesses and control flow. We target the following optimization levels: `-O0`, `-O1`, `-O2`, `-O3`, `-Ofast`, `-Os`, `-Oz`³.

Finally, we also tested 6 processor architectures: x86-64, x86-i386, armv7, aarch64, RISC-V⁴, and MIPS-32. We believe that this list represents a good mix of ISAs used for all kinds of computing systems. We also note that during the disclosure, the contacted library developers considered all these architectures as important.

All target libraries usually do not support such a large variety of compilers, optimization levels, and architectures. We developed some best-effort compilation scripts to be able to compile for all of these targets. However, some combinations of parameters either lead to a non-trivial error during compilation, a broken binary, or the resulting binary became so inefficient that our analysis ran out of memory on the smaller machines (mainly with `-O0`).

Removing Source-Code Issues. Even though constant-time programming has been known for a long time, some of the targeted libraries still contain secret-dependent issues that are rooted in source code, e.g., AES implementations with lookup tables. Such issues are usually not removed by the compiler and thus also show up in our results. We target compiler-induced issues specifically and, therefore, need to filter any such issues. To achieve that, we maintain a manually curated list of functions or files that contain timing issues that originate from the source code and remove any

issue that links back to this list. However, such filtering has its limitations: it does not always work due to compilers inlining functions or other optimizations that remove or modify the connection to the source code, and our manually curated list might miss some source-code issues and not remove them from our dataset. On the other hand, removing entire functions might also remove some compiler-induced issues within these functions. Nevertheless, for some libraries and compilers, it was impossible to filter results by function name as the compiler often removes this information. We manually verify that the other secret dependencies flagged by our tool stem from source code that seems to follow defensive programming guidelines for cryptographic code [5].

Automation. To run such a wide-ranging study, we developed an experimentation framework [41] based on Kubernetes. Our platform takes a set of parameters to evaluate as input, e.g., target the library BoringSSL with all supported compilers, all optimization levels, all architectures, and all algorithms. It then launches individual jobs for each combination (target algorithms are merged into the same job). Each job first sets up the required compiler and downloads the specified version of the target library. Then, it builds the library with the selected optimization level. Finally, our tool emulates the binary and collects traces for the selected cryptographic algorithms. Our tool then generates the reports and saves them to a shared network drive.

5 Analysis

In this section, we quantitatively analyze how compiler parameters and libraries lead to compiler-induced secret-dependent operations. We first discuss how many secret-dependent operations were flagged and if there are differences between the processor architectures, compilers, compiler versions, and optimization levels. From the total of 44,604 experiments that we ran, 37,818 (84.8%) were completed successfully. The remaining experiments failed either due to compiler errors or due to our system running out of memory during the address trace collection. Of the total number of 6,608 binaries, we compiled 5,716 successfully (86.5%). Our experiments span 54 implementations of cryptographic primitives.

5.1 Found Issues

In the 5,716 binaries, we found at least one secret-dependent control-flow operation in 429 (7.5%) and at least one secret-dependent

³The optimization level `-Oz` is only available in LLVM and is thus omitted for GCC.

⁴RISC-V is only supported starting from GCC version 10 and LLVM version 10 onwards

Table 1: The target cryptographic libraries and their respective support of constant-time cryptographic primitives. We use ✓ and ✗ to indicate claimed constant-time and non-constant-time implementations respectively. Implementations that use blinding are indicated using * and handwritten assembly implementations using $\mathbb{Z}$.

	HACL*	Libsodium	Botan	BearSSL	BoringSSL	OpenSSL	WolfSSL	MbedTLS
AES CBC	$\mathbb{Z}$	✗	✓	✓	✓	✗	✓	✗
AES CTR	$\mathbb{Z}$	✗	✓	✓	✓	✗	✓	✗
AES GCM	$\mathbb{Z}$	$\mathbb{Z}$	✓	✓	✓	✗	✓	✗
Chacha20Poly1305	✓	✓	✓	✓	✓	✓	✓	✓
HMAC SHA1	✓	✗	✓	✓	✓	✓	✓	✓
HMAC SHA256	✓	✓	✓	✓	✓	✓	✓	✓
HMAC Blake2	✓	✗	✓	✗	✓	✓	✗	✗
Curve25519	✓	✓	✓	✓	✓	✓	✓	*
ECDH P256	✓	✗	*	✓	✓	*	*	*
ECDSA P256	✓	✗	*	✓	✓	*	*	*
RSA	✓	✗	*	✓	*	*	*	*

memory access in 368 (6.4%). In absolute numbers, we found 13,341 compiler-induced issues in the investigated libraries. We list the number of issues per library in Table 2. We note that the number of issues found is exaggerated because the same issue can appear in multiple places in a binary or trace (e.g., due to inlining a function). For example, in Botan’s implementation of x25519, we found a total of 5,499 issues, 4,221 of which are found in binaries for MIPS. A brief manual analysis of the issues in Botan’s x25519 implementation on MIPS shows the same issues popping up more than 100 times for every experiment. This wealth of issues can overshadow other more nuanced issues that do not appear that often. Therefore, in some of the following analyses, we simply consider the following four cases: (1) we did not find a secret dependency, (2) we found at least one control-flow secret dependency, (3) we found at least one secret-dependent memory access, and (4) we found both secret-dependent control-flow and memory accesses.

5.2 Processor Architectures

We depict all binaries with at least one control flow and memory accesses that were flagged to be secret dependent in Figure 2. In particular, there is a significant difference between processor architectures. We found more binaries with secret dependencies on less commonly used processor architectures, i.e., MIPS, RISC-V, and x86-i386. This is especially notable for RISC-V, which is only supported in 8 out of 17 compiler versions and thus has less than half the experiments of other architectures. Nevertheless, we found more binaries with at least one secret-dependent operation in RISC-V compared to x86-64, armv7, and aarch64. More mainstream processor architectures seem to be less affected but not entirely free of compiler-induced issues.

5.3 Compilers and Optimization Levels

We depict the number of binaries in which some issues were flagged according to the compiler and the version that produced it in Figure 3. Interestingly, older compilers seem to introduce fewer issues. We believe this is due to more aggressive optimizations in newer compilers, potentially undoing defensive programming

Table 2: Number of compiler-induced secret dependent operations found per target library across all parameters. Note that the number of reported issues may appear very large due to finding the same code patterns multiple times.

Library	Crypto Primitive	Flagged Issues
BearSSL	rsa	400
	ecdh-p256	61
	ecdsa	46
BoringSSL	ecdsa	120
	ecdh-p256	104
Botan	curve25519	5000
	chacha-poly1305	336
	aes-gcm	33
	hmac-blake2	26
	hmac-sha1	26
	hmac-sha2	26
	aes-ctr	26
	aes-cbc	26
HACL*	rsa	6729
	ecdh-p256	244
	ecdsa	136
Libsodium	curve25519	2
Total		13,341

techniques. Some of these optimizations are discussed in the following section (Section 6), where we manually analyze individual binaries.

We found binaries with issues in all optimization levels as depicted in Figure 3. We observe notable drops for LLVM’s and GCC’s -O0. We believe that fewer enabled optimizations also lead to fewer compiler-induced issues. We discuss in Section 7 that optimizations are one of the reasons compilers introduce side channels;

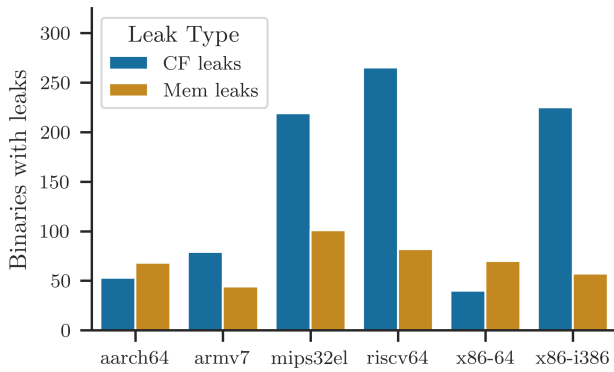


Figure 2: Binaries with at least one control-flow or memory-based secret-dependent operation by architecture. We observe more leaks in less popular processor architectures.

thus fewer optimizations could mean defensive programming techniques are less likely to get interfered with. However, we note that this cannot be considered a solution to compiler-induced issues as we still observe some issues with `-O0`. We also note that the `-O0` results may be misleading since we could not analyze some algorithms in some libraries with `-O0` as our machines ran out of memory. Therefore, the experiments with `-O0` are not as thorough as the other optimization levels. We also observe a reduction in flagged issues with `-Os`. It is unclear why these differences exist.

6 Notable Findings

We perform an in-depth manual analysis for a subset of the found secret-dependent operations. Here, we discuss some notable findings by showing the portable source code snippet and the corresponding binary introducing the secret dependency. If possible, we also briefly analyze the underlying mechanism in the compiler that introduces the secret dependency. We only discuss the simplest examples or those that we can minimally reproduce in this section.

6.1 HACL*

Secp256. In HACL*, we discover that the compiler can introduce secret-dependent control flow into the implementation of the elliptic curve *secp256*. This secret dependency is executed in HACL*'s implementation of ECDH-p256 and ECDSA, as both rely on the *secp256* implementation. The code snippet that leads to the secret dependency can be found in Listing 2. The intended functionality of the function *cmovznz* is a conditional move: it moves either x or y to r depending on a secret *condition* (in constant time). The function achieves this by first computing a *mask* of either all 1's or all 0's depending on the condition (line 4 in Listing 2a). Afterwards, r is set to $(y \wedge \text{mask}) \vee (x \wedge \neg \text{mask})$. Thus setting r to either x or y depending on the *mask* and implicitly the *condition* (line 5 and 6 in Listing 2a). One would expect this to be compiled into some binary AND and OR operations. However, some versions of LLVM understand that this is the same as a branch and

replace the bit-wise operations. On MIPS and armv7, LLVM replaces the bitmask with a direct calculation of the address of the selected value to be stored in r . We depict an example resulting armv7 binary in Listing 2b. On both MIPS and armv7 we observed this behavior for versions 14 and 15 and optimization levels `-Os` and `-Oz`. On RISC-V and x86-i386, LLVM replaces the bitmask operations with four control flow instructions. We observe this behavior on LLVM versions 13, 14, and 15 on all optimization levels besides `-O0`.

Further investigation shows that LLVM understands the bitmask generation and arithmetic logic and replaces it with a select IR instruction with the semantic of assigning a variable to one of the parameters depending on the input condition. The culprit of this optimization is a pass called *InstCombine*, which performs optimizations by combining and simplifying arithmetic operations. In this case, *InstCombine* replaces the mask generation and the subsequent bitwise AND and ORs with a select. According to the LLVM documentation [30], this is a potential optimization to allow the backend to decide how to instantiate the select IR instruction based on the processor architecture. For example, on x86-64, select is sometimes instantiated with a *cmov* instruction.

6.2 Botan

AES-GCM. We discover compiler-induced secret-dependent control flow in Botan's implementation of GHASH used in AES-GCM. Listing 3 shows a simplified snippet of how Botan precomputes the H-values for the key schedule of GHASH in a supposedly purely arithmetic way. However, instead of just performing the XOR operation with carry, LLVM 15 replaces this with a branch on RISC-V: If carry is 0, then jump over the XOR.

128-bit Custom Type. We discover a similar compiler-induced secret-dependent operation in the implementation of a custom 128-bit wide type in Botan used in Chacha-Poly1305 and x25519. For platforms that do not natively support 128-bit integers, Botan uses a 128-bit software implementation that uses two 64-bit words. Botan implements the addition operator by first adding the lower half, calculating the carry, and then adding the upper half and the carry. We depict Botan's implementation in Listing 4. Similar to the previous issue in Botan's GHASH implementation, binaries by some compilers first check the carry and skip the second addition if the carry is not set. We observe this behavior for GCC on MIPS and x86-i386 under all optimization levels (including no optimization `-O0`). Interestingly, the compiler can generate both secret-dependent control flow and memory accesses at the same time. For example, using GCC 11.3.0 on MIPS with `-O2` produces a binary with memory and control-flow secret dependencies. We observe this operator getting called in Botan's x25519 and chacha-poly1305 implementation.

6.3 BearSSL

Secp256. BearSSL's implementation of the elliptic curve *secp256* contains a small routine to check a scalar; a simplified version is shown in Listing 5. In this function, a scalar is checked to be below the curve order. This is achieved by comparing the scalar byte-by-byte to the curve order (*P256_N*). Recent versions of LLVM for RISC-V replace the bitmask arithmetic operations with a secret-dependent branch.

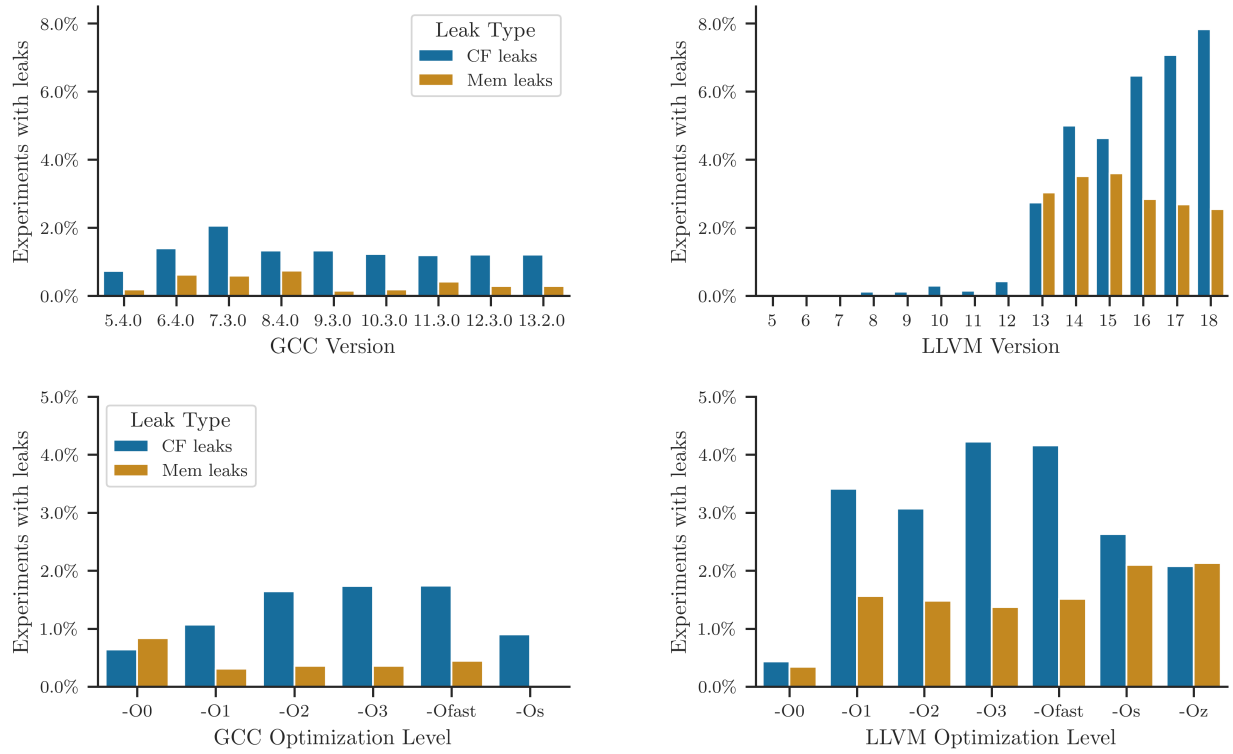


Figure 3: Experiments with at least one secret-dependent operation per compiler, respective versions, and optimization level.

```

1 void cmovznz4(uint32_t condition, uint32_t *x,
2             uint32_t *y, uint32_t *r) {
3     uint32_t mask = ~calc_mask(condition);
4     r[0] = (y[0] & mask) | (x[0] & ~mask);
5 }

```

(a) Source code snippet.

```

1 cmovznz4:
2     cmp     r0, #0
3     movne  r1, r2
4     ldr     r0, [r1]
5     str     r0, [r3]
6     bx     lr

```

(b) armv7 assembly (LLVM 15, -Os).

Listing 2: Simplified snippet used in HACL*’s implementation of elliptic curve P256. This function implements a conditional move, i.e., it moves either x or y to r depending on condition. The function `calc_mask` creates a bitmask based on `condition` with either all 1s or all 0s. LLVM replaces the bitmask arithmetic with a `movne` to compute the address followed by secret-dependent memory accesses in armv7 and MIPS. In RISC-V and x86-i386, the bitmask arithmetic gets replaced with a secret-dependent branch instead.

Modular Exponentiation. We discover another compiler-induced secret-dependent operation in BearSSL’s implementation of modular exponentiation. The source code snippet and the corresponding

```

1 uint64_t precompute(uint64_t H0, uint64_t H1) {
2     const uint64_t R = 0xE100000000000000;
3     const uint64_t carry = R * (H1 & 1);
4     return (H0 >> 1) ^ carry;
5 }

```

(a) Simplified source code.

```

1 precompute:
2     andi   %a1, %a1, 1
3     srli   %a0, %a0, 1
4     beqz   %a1, .LBB0_2
5     li     %a1, -31
6     slli   %a1, %a1, 56
7     xor    %a0, %a0, %a1
8 .LBB0_2:
9     ret

```

(b) RISC-V assembly (LLVM 15, -O2).

Listing 3: Simplified source code snippet found in Botan’s implementation of GHASH used in AES-GCM. This function is a part of the key schedule. The compiler produces a binary that skips the addition if the carry is 0.

x86-64 assembly can be found in Listing 6. The desired functionality of the snippet is similar to the `cmovznz` snippet described above for HACL*. The snippet either sets x to y or to z depending on the condition. EQ computes a mask of either all 0’s or all 1’s depending if `condition == 0`. LLVM on Aarch64 and x86-64 replace the


```

1 uint128_t operator+=(uint128_t &x) {
2     l += x.l; h += x.h;
3     uint64_t carry = (l < x.l);
4     h += carry;
5     return *this;
6 }

```

Listing 4: Simplified source code snippet found in Botan’s implementation of a 128-bit type used in chacha-poly1305. The compiled binary skips the carry addition if the carry is 0.

```

1 uint32_t check_scalar(const unsigned char *k) {
2     uint32_t c = 0;
3     for (int u = 0; u < 32; u++) {
4         c |= CMP(k[u], P256_N[u]);
5     }
6     return c;
7 }

```

Listing 5: Simplified check-scalar routine in BearSSL’s implementation of secp256. This routine checks if the key is lower than the curve order. The function CMP compares two numbers and returns -1, 0, 1 if the first argument is smaller, equal, or larger using binary arithmetic operations. Recent LLVM versions on RISC-V detect the bitmask arithmetic and replace it with a secret-dependent branch.

mask computation with a conditional mov on x86-64 and a conditional select on aarch64, respectively. However, the compiler does not use this instruction to set the x variable directly. It uses it to compute an address, which is then loaded and stored in x. This leads to a secret-dependent memory access operation which leaks the condition as either the memory location of y or z is read and written into x. LLVM performs a part of this optimization in the InstCombinePass where the bitmask computation is replaced with a select IR instruction (similar to HACL*). In the backend, the compiler then replaces the select with a cmov or csel instruction, which computes the address for the secret-dependent memory access.

6.4 BoringSSL

Complex Branching Condition. Recent LLVM compilers may split a branch with complex conditions into multiple branches with parts of the condition in a snippet found in BoringSSL. This behavior creates a small leak because which branch is taken leaks which part of the condition was true or false. We observed this behavior in the point-add routing of P256 on armv7, MIPS, and x86-i386. A snippet that shows the culprit branch is depicted in Listing 7.

7 Problematic Compiler Internals

We categorize the found compiler-induced secret-dependent operations into three types: replacing bitmask arithmetic with branches, arithmetic shortcuts to skip computation, and splitting complex branching conditions into multiple branches. The first issue was previously studied in toy examples [42], but to the best of our knowledge, the latter two have not yet been discussed. We analyze how and why these optimizations are introduced in modern

```

1 uint64_t mask1 = ~(uint64_t)EQ(condition, 0);
2 uint64_t mask2 = ~mask1;
3 for (int i = 0; i < len; i++) {
4     x[i] = (mask1 & y[i]) | (mask2 & z[i]);
5 }

```

(a) Source code snippet.

```

1 test    %ecx, %ecx
2 lea     %r8, [%rdi + 8*%r9]
3 lea     %rax, [%rdx + 8*%r9]
4 cmov    %rax, %r8
5 mov     %rax, qword ptr [%rax]
6 mov     qword ptr [%r8], %rax

```

(b) x86-64 assembly (LLVM 15, -Os).

Listing 6: Simplified source code snippet in BearSSL. Similar code patterns are used in its modular exponentiation implementations used in RSA, ECDH-P256, and ECDSA. Recent versions of LLVM on x86-64 replace the mask with a cmov instruction to create the address that is used to write a value into x[i]. This thus leaks the condition as the address of y[i] is different to z[i]. On Aarch64, LLVM inserts csel instead of cmov.

```

1 uint32_t is_nontriv_double = ct_is_zero(xneg | yneg) &
2                               ~ct_is_zero(z1nz) &
3                               ~ct_is_zero(z2nz);
4 if (is_nontriv_double) {
5     fiat_p256_point_double(x3, y3, z3, x1, y1, z1);
6 }

```

Listing 7: Simplified source code snippet in BoringSSL. Recent versions of LLVM on armv7, MIPS, and x86-i386 insert two distinct branching instructions instead of a single branch. Which of the two branching instructions is taken leaks information to an attacker and depends on the secret.

compilers. We note that this list is not comprehensive, as more compiler-induced secret dependencies might appear in future compilers. Since these code patterns are also used in other libraries without any introduced secret-dependent operations, we analyze whether these patterns can be used safely.

Simplification of Bitmask Arithmetic. LLVM seems to be very advanced in understanding and optimizing bitwise arithmetic operations, as we found most such issues in binaries produced by LLVM. However, we also found some simplifications in binaries compiled by GCC, albeit less than with LLVM. In the following, we focus on LLVM as it optimizes bitmask arithmetic more aggressively.

Compilers do numerous optimization passes on the input program. In the context of bitmask arithmetic, LLVM contains an optimization pass that tries to reduce redundant instructions called instcombine [30]. This pass tries to perform algebraic simplifications by employing a big recursive pattern-matching engine. In the observed cases, the instcombine pass replaces the mask generation and the subsequent bitwise AND and ORs with a select LLVM instruction. The LLVM compiler instantiates the select instruction in the ISA-specific backend based on the processor architecture. For

example, on x86-64, `select` is sometimes instantiated with a `cmov` instruction, and on RISC-V and MIPS with conditional branches. Sometimes, the ISA-specific backend removes the `select` instruction altogether and does not introduce secret-dependent operations.

Since bitmask arithmetic is a popular building block in constant-time cryptographic implementations, we also investigate how other libraries leverage bitmasks without producing secret-dependent operations. Many libraries use the same implementation of various cryptographic algorithms, such as `poly1305-donna` [34]. This implementation of `poly1305` contains a bitmask arithmetic pattern in the function `poly1305_finish`. In our experiments, no compiler produced a binary with secret-dependent operations for this implementation. We do not know if the current source code pattern is deliberately chosen to avoid secret-dependent operations or if this is just a lucky coincidence. For now, this implementation seems not to produce secret-dependent operations with the evaluated compilers. To test the robustness of code patterns using bitmask arithmetic, we randomly tried innocent source code changes and observed that compilers once again introduce secret-dependencies⁵. This highlights that bitmask arithmetic is not very robust and may unexpectedly produce secret-dependent operations. We did not find a way to ascertain if a specific arithmetic source code pattern is safe just by looking at the source code.

Arithmetic Shortcuts. Arithmetic shortcuts use branches to skip further computation (c.f., Listing 3). We observed this behavior with both GCC and LLVM. Further manual experiments with source code patterns show that this optimization is mainly performed in the ISA-specific backend of the two compilers. We believe the issues found in this study to be misguided optimizations. Skipping an XOR instruction will be slower than just performing the XOR as it is a very fast instruction. In the instances found in this study, the compiler’s frontend injects the secret-dependent branch via a `select` IR instruction, similar to the bitmask arithmetic optimization above. Since all architectures share the same frontend in LLVM, they all have a `select` instruction in their IR before the backend optimizations. Some backends for less popular architectures, such as MIPS and RISC-V, produce inefficient binaries that use branches to skip an XOR operation in some LLVM versions. Luckily, backends for the other four architectures remove the secret-dependent `select` again and just perform the XOR.

To further experiment with this behavior, we manually create a code snippet that would greatly benefit from optimization: we created a function with a boolean input that allows a division operation to be skipped if it is true. Advanced compilers know that division is an expensive operation and, therefore, produce a binary that skips the division if possible (e.g., when the input is true). We managed to reproduce this with LLVM in arguably the most popular architecture, x86-64, where an ISA-specific pass introduces a branch that skips the division.

In summary, branches to skip expensive computations are a common optimization in both compilers. They are instrumental in optimizing performance in cases where the expensive operation is often unnecessary and can be skipped. We observed these optimizations in multiple places in modern compilers: in the shared frontend

and the architecture-specific backend of the compiler. Current issues that we found seem to be misguided and can be fixed with newer compilers with better optimizations. However, we found other patterns that did not occur in the surveyed libraries where secret-dependent branches would be injected. Therefore, developers should be careful in cases where an expensive operation could be skipped for 50% of the inputs.

Complex Branch Conditions. In some cases, compilers can split branches with complex conditions into multiple branching instructions, each with a part of the condition (c.f., Listing 7). This behavior allows an adversary to leak which branching instruction was taken and thus learn some information about the complex condition. The developer might assume that the condition is checked atomically, but the compiler gives no such guarantee. For the same architecture and compilation flags, sometimes a source code pattern is safe. However, just slight tweaks to the code (still adhering to the same pattern) lead to the compiler introducing secret-dependent operations. Therefore, avoiding such compiler-induced secret dependencies is very hard without binary analysis. However, we have only rarely found this behavior and only in LLVM.

8 Takeaways

Defensive programming techniques can be ineffective. As we have shown in the previous sections, cryptographic libraries often contain compiler-induced secret dependencies even though they apply current defensive programming techniques. The manual analysis of the flagged issues indicates that the issues that we find are not due to developer error but rather unexpected transformations by the compiler. We thus conclude that the current defensive programming techniques are insufficient. It remains unclear if the rules are incomplete, i.e., a new specific rule would lead to safe source code again, or if they are fundamentally broken with modern compilers. We defer the exploration of such new rules to future work.

Constant-time source code is brittle. As seen in our analysis of findings in Section 6, compilers apply very smart simplifications and optimizations to source code. However, we also demonstrated that some of these seem to introduce side-channel vulnerabilities in some implementations. The fact that only some implementations suffer from this suggests that compilers currently only optimize certain code patterns in “dangerous” ways. Avoiding such patterns is not a definite solution, though: the same code patterns that, for now, seem to be fine based on our analysis might be broken by the next generation of compilers. For example, LLVM started to aggressively simplify bit-wise mask arithmetic to select IR instructions starting from version 12. Our results indeed show a spike in the number of found vulnerabilities from that version in Figure 3. For now, LLVM only simplifies some types of bitmasks dangerously, and it does not recognize others – future improvements might change this and lead to even more such issues.

Recommendations for libraries are that new releases should come with a list of tested target architectures, operating systems, compilers, and flags. Then, users of a library can at least use a compiler that is believed to maintain the constant-time properties of the library. Nevertheless, our work demonstrates how brittle the current state of constant-time high-level source code has become.

⁵E.g., we only moved the `select` bitmask part to its own function.

The Case for Continuous Testing. Some of the issues that we discovered are produced by compilers that are up to 10 years old. This is a significant time period, and it is unclear why such behavior was not flagged before. We believe that rigorous continuous testing on as many processor architectures and compilers as possible should be the standard for cryptographic libraries. A recent survey [21] has asked developers of cryptographic libraries about the tools they use and highlights that most libraries still lack such infrastructure. Our findings also confirm this. We approached the maintainers of the libraries targeted in this paper to offer our help if they want to add our tool to their infrastructure. Some of them were receptive to this idea, and we are currently evaluating if this is feasible.

9 Defenses

Principled Defenses. There are at least two principled defenses that eliminate all compiler-induced secret dependencies: formal analysis of the *binary* or using a compiler that guarantees not to include any issues.

Formal verification of assembly is used on the predominant processor architectures [10] only because it requires a high degree of effort and know-how of the targeted processor architecture. Nevertheless, such code is guaranteed to remain constant-time as it does not go through any other transformation.

The second principled defense is using a compiler that guarantees no such induced issues. Ideally, the compiler comes with formal guarantees that the behavior of the source code in constant time will not be changed, such as CompCert [7, 8]. However, such compilers are rarely used in practice as they lag behind commodity compilers in supported architectures and performance. There have also been many proposals to add passes to commodity compilers or to annotate functions or variables to disable certain optimizations [12]. However, these efforts have not yet had the desired effect on mainstream compilers and have not been included upstream.

Compilers that transform secret-dependent code into constant-time binaries [11, 38] usually transform general-purpose code into a linear binary that executes all branches no matter the input. These approaches have been shown to produce side-channel free binaries [11], but they are usually research artifacts and, as such, not yet usable for production.

Certification. Another option is certification: Cryptographic libraries could use tools to certify any binary produced by a commodity compiler, thus guaranteeing that the binary does not contain secret-dependent operations. Formal or symbolic tools could prove the absence of secret-dependent operations in binaries. We note that the approach used in this paper cannot be used to provide this guarantee, as we only find such issues but cannot prove the absence thereof. Certifying binaries notably shifts the release procedure of major libraries as they should no longer release the source code to be compiled but rather multiple binaries for different architectures and operating systems. This would require a significant effort for every single release. Obviously, an important aspect of these libraries is the fact that they are open source, thus binary certification would be most beneficial in conjunction with approaches akin to reproducible builds.

Increased Testing. Some libraries already perform in-depth testing prior to a new release. For instance, libsodium [14] performs tests and analysis for more than 20 target platforms: from WebAssembly on V8, various compilers for x86-64 Ubuntu and macOS, to Debian on SPARC. As we did not find many issues in libsodium, this rigorous testing seems to have a beneficial effect. However, we want to note that libsodium is the smallest library that we test with the most limited support for cryptographic primitives. For example, libsodium does not support the elliptic curve secp256; instead, they do all of their elliptic curve operations on curve25519. Therefore, it may be unfair to compare to larger libraries with support for more cryptographic primitives.

We remark that increased testing does not provide any formal guarantee of the binary compared to certification. Nevertheless, rigorous testing can help mitigate compiler-induced issues and detect them before they can do any harm.

10 Related Work

This paper presents contributions in two main directions: i) a new tool that allows testing for side-channels in many architectures from a single platform, and ii) a study of dangerous compiler transformations in constant-time libraries. We stress that the literature is rich in terms of related tools, and while its novelty is nuanced, as we describe below, its main advantage is that it enables performing such a study.

Side-Channel Detection Tools. Numerous related tools detect side channels in various targets, from static analysis of higher-level source code to statistical tests of the final binaries. For an extensive survey of such tools, we refer to [6] and [21].

Statistical test tools collect numerous black-box measurements and use statistical tools to interpret the measurements. For example, dueduct [39] uses Welch’s t-test to check for information leakage. While any leaks found by such tools are usually immediately exploitable, they also miss a significant number of leaks as they only consider their target as a black box. In practice, side-channel attacks have become much more powerful with improved granularity and synchronization [33, 45].

Symbolic trace-based tools [15] use symbolic execution to evaluate all paths of a target. CacheAudit [15], one of the most well-known tools in the category, uses these traces to provide an upper bound for the leakage. It is easy to reason about a zero upper bound, but any other value is hard to interpret, as it does not help to pinpoint the issue. However, these tools are usually tailored to one processor architecture and cannot easily be extended to others.

Formal analysis tools provide strong guarantees as they can prove the absence of issues in a target [3, 7, 50]. However, these tools usually only reason on high-level source code or some intermediate representation. It is unclear if and how these guarantees translate to binaries⁶.

Dynamic tools enable the analysis of the actual execution of the final binary. For example, ctgrind [25] uses Valgrind to trace hand-annotated secret memory regions and report any secret-dependent access to it. The closest related tools to the one developed for our

⁶If a formally proven compiler [9] is used, then the guarantees should translate down to the binary.

study are trace-based dynamic analysis tools [19, 46, 48]. Trace-based approaches record a number of traces for different secret inputs and then compare the traces to identify secret-dependent operations. These approaches have various advantages: they are relatively performant, reason about the final binary, and can precisely pinpoint the issues. However, they are incomplete: they cannot guarantee the absence of leaks. A final limitation of these tools is their limited portability, as they usually rely on binary instrumentation frameworks that are not portable. As such, these tools usually only support a single processor architecture [47].

Automatically Fixing Secret Dependencies. This paper focuses on libraries that are already hardened against all side-channel attacks. However, the vast majority of programs are not hardened. Various approaches that transform non-hardened code to a binary that contains no secret dependencies exist [11, 38]. However, we note that these approaches are not used by any security-critical library that we investigated. Nevertheless, our analysis could be extended to binaries produced by these approaches.

Related Studies. Compiler-induced secret-dependent operations have been studied in the past. They have been reproduced using toy examples [13, 42]. The toy examples mostly focus on binary arithmetic and how compilers may simplify these. Binsec/Rel [13] also performs a study of several constant-time cryptographic implementations on the binary level on x86 and arm architectures. However, they do not investigate other architectures nor how other compilers can affect these implementations. To the best of our knowledge, we are the first to demonstrate compiler-induced issues in constant-time cryptographic libraries in the wild.

Kaufmann et al. discovered a secret dependency in a curve25519 implementation [22]. They detect that the Windows runtime library introduces a secret-dependent variation using the compiler MSVC 2015. In this paper, we do not trace the runtime library and will not flag such issues. All of the secret dependencies that we found purely originate from the library itself. We remark that issues in underlying runtime libraries are no less critical, and our approach could also be extended to also trace them.

11 Conclusion

General-purpose compilers can interfere with hardening techniques used to write high-level constant-time code, e.g., in cryptographic libraries. In this paper, we investigated whether this happens in the wild and to what extent. For this, we developed a flexible pipeline based on trace-based dynamic analysis that allowed us to measure a large number of target libraries, compilers, optimization versions, and processor architectures. Using our pipeline, we analyzed whether the resulting binaries and algorithms exhibited any of the following two behaviors: control-flow decisions and memory accesses based on secret values, both of which are symptomatic of non-constant-time execution.

Our study demonstrates that compiler-induced side-channel vulnerabilities are widespread in cryptographic primitives of libraries that adopt constant-time programming techniques. We observed issues in all tested processor architectures, compiler versions, and optimization levels. These issues emerged in 5 of the 8 profiled libraries – even one where the source code was formally verified

to be free of such side channels. We analyze a few notable cases in detail: the main secret-dependent operations introduced by the compilers are arithmetic shortcuts to skip some computation (e.g., early exit from loops), replacing bitmask arithmetic with branches, and splitting complex branching conditions – directly reversing the recommendations of defensive programming guidelines to achieve constant-time code [5]. This suggests that compilers silently “undo” the code patterns that developers introduce purposefully to ensure constant-time execution.

Current principled defenses do not seem ready for mass adoption: compilers that provide constant-time code guarantees are specialized and lag behind general-purpose ones in terms of supported architectures, features, and performance. Formally verifying the assembly produced by compilers requires high manual efforts and is thus reserved for the most popular architectures only – confirmed by our findings that less popular architectures are more subjected to compiler-induced vulnerabilities. Distributing certified binaries would greatly increase the efforts of developers and users of libraries alike.

Our work highlights that current defensive programming techniques are more akin to anecdotal workarounds to compilers’ behavior rather than a systematic solution. Our results point out that high-level constant-time code is prone to dangerous transformations and optimizations from compilers and calls for cooperation between developers of security-critical libraries and general-purpose compilers.

Acknowledgments

This work was supported by the Zurich Information Security and Privacy Center, and ETH Future Computing Laboratory (financed by a donation from Huawei Technologies).

References

- [1] Onur Acıçmez, Çetin Kaya Koç, and Jean-Pierre Seifert. 2006. Predicting Secret Keys Via Branch Prediction. In *Topics in Cryptology – CT-RSA 2007*, Masayuki Abe (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 225–242.
- [2] Martin R. Albrecht and Kenneth G. Paterson. 2016. Lucky Microseconds: A Timing Attack on Amazon’s s2n Implementation of TLS. In *Advances in Cryptology – EUROCRYPT 2016*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 622–643.
- [3] J Bacerlar Almeida, Manuel Barbosa, Jorge S Pinto, and Bárbara Vieira. 2013. Formal verification of side-channel countermeasures using self-composition. *Science of Computer Programming* 78, 7 (2013), 796–812.
- [4] Jean-Philippe Aumasson. 2012. A cryptography coding standard. https://web.archive.org/web/20190203024646/https://cryptocoding.net/index.php/Coding_rules.
- [5] Jean-Philippe Aumasson. 2019. Cryptocoding. <https://github.com/veorq/cryptocoding>.
- [6] M. Barbosa, G. Barthe, K. Bhargavan, B. Blanchet, C. Cremers, K. Liao, and B. Parno. 2021. SoK: Computer-Aided Cryptography. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 777–795. doi:10.1109/SP40001.2021.00008
- [7] Gilles Barthe, Gustavo Betarte, Juan Campo, Carlos Luna, and David Pichardie. 2014. System-Level Non-Interference for Constant-Time Cryptography. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (Scottsdale, Arizona, USA) (CCS ’14)*. Association for Computing Machinery, New York, NY, USA, 1267–1279. doi:10.1145/2660267.2660283
- [8] Gilles Barthe, Sandrine Blazy, Benjamin Grégoire, Rémi Hutin, Vincent Laporte, David Pichardie, and Alix Trieu. 2019. Formal Verification of a Constant-Time Preserving C Compiler. *Cryptology ePrint Archive*, Paper 2019/926. doi:10.1145/3371075 <https://eprint.iacr.org/2019/926>.
- [9] G. Barthe, B. Gregoire, and V. Laporte. 2018. Secure Compilation of Side-Channel Countermeasures: The Case of Cryptographic “Constant-Time”. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE Computer Society, Los Alamitos, CA, USA, 328–343. doi:10.1109/CSF.2018.00031

- [10] Barry Bond, Chris Hawblitzel, Manos Kapritsos, K. Rustan M. Leino, Jacob R. Lorch, Bryan Parno, Ashay Rane, Srinath Setty, and Laure Thompson. 2017. Vale: Verifying High-Performance Cryptographic Assembly Code. In *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Vancouver, BC, 917–934. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/bond>
- [11] Pietro Borrello, Daniele Cono D'Elia, Leonardo Querzoni, and Cristiano Giuffrida. 2021. Constantine: Automatic Side-Channel Resistance Using Efficient Control and Data Flow Linearization. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, Republic of Korea) (CCS '21)*. Association for Computing Machinery, New York, NY, USA, 715–733. doi:10.1145/3460120.3484583
- [12] Sunjay Cauligi, Gary Soeller, Brian Johannesmeyer, Fraser Brown, Riad S. Wahby, John Renner, Benjamin Grégoire, Gilles Barthe, Ranjit Jhala, and Deian Stefan. 2019. FaCT: A DSL for Timing-Sensitive Computation. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (Phoenix, AZ, USA) (PLDI 2019)*. Association for Computing Machinery, New York, NY, USA, 174–189. doi:10.1145/3314221.3314605
- [13] Lesly-Ann Daniel, Sébastien Bardin, and Tamara Rezk. 2020. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary-Level. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*. IEEE, 1021–1038. doi:10.1109/SP40000.2020.00074
- [14] Frank Denis. [n.d.]. libsodium. <https://github.com/jedisct1/libsodium>
- [15] Goran Doychev, Boris Köpf, Laurent Mauborgne, and Jan Reineke. 2015. Cacheaudit: A tool for the static analysis of cache side channels. *ACM Transactions on Information and System Security (TISSEC)* 18, 1 (2015), 1–32.
- [16] Nicolas Dutly and Moritz Schneider. [n.d.]. Microsurf. <https://github.com/Jumpst3r/microsurf>.
- [17] Rich Felker. [n.d.]. musl libc. <https://musl.libc.org/>.
- [18] Google. 2014. BoringSSL. <https://boringssl.googlesource.com/boringssl/>
- [19] Shaobo He, Michael Emmi, and Gabriela F. Ciocarlie. 2020. ct-fuzz: Fuzzing for Timing Leaks. In *13th IEEE International Conference on Software Testing, Validation and Verification, ICST 2020, Porto, Portugal, October 24-28, 2020*. IEEE, 466–471. doi:10.1109/ICST46399.2020.00063
- [20] Intel. 2007. Intel Pin - A dynamic binary instrumentation framework. <https://www.intel.com/content/www/us/en/developer/articles/tool/pin-a-dynamic-binary-instrumentation-tool.html>.
- [21] J. Jancar, M. Fourné, D. Braga, M. Sabt, P. Schwabe, G. Barthe, P. Fouque, and Y. Acar. 2022. “They’re not that hard to mitigate”: What Cryptographic Library Developers Think About Timing Attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 632–649. doi:10.1109/SP46214.2022.9833713
- [22] Thierry Kaufmann, Hervé Pelletier, Serge Vaudenay, and Karine Villegas. 2016. When Constant-Time Source Yields Variable-Time Binary: Exploiting Curve25519-donna Built with MSVC 2015. In *Cryptography and Network Security*, Sara Foresti and Giuseppe Persiano (Eds.). Springer International Publishing, Cham, 573–582.
- [23] Paul Kocher, Joshua Jaffe, and Benjamin Jun. 1999. Differential Power Analysis. In *Advances in Cryptology — CRYPTO’ 99*, Michael Wiener (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 388–397.
- [24] Paul C. Kocher. 1996. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In *Advances in Cryptology — CRYPTO ’96*, Neal Koblitz (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 104–113.
- [25] Adam Langley. 2010. Checking that functions are constant time with Valgrind. <https://www.imperialviolet.org/2010/04/01/ctgrind.html>.
- [26] Kaijern Lau and Anh Quynh Nguyen. [n.d.]. Qiling Framework. <https://github.com/qilingframework/qiling>.
- [27] Sangho Lee, Ming-Wei Shih, Prasun Gera, Taesoo Kim, Hyesoon Kim, and Marcus Peinado. 2017. Inferring Fine-grained Control Flow Inside SGX Enclaves with Branch Shadowing. In *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Vancouver, BC, 557–574. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/lee-sangho>
- [28] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee. 2015. Last-Level Cache Side-Channel Attacks are Practical. In *2015 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 605–622. doi:10.1109/SP.2015.43
- [29] Jack Lloyd. 2000. Botan: Crypto and TLS for Modern C++. <https://botan.randombit.net/>
- [30] LLVM. [n.d.]. LLVM Language Reference Manual. <https://llvm.org/docs/LangRef.html>.
- [31] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: building customized program analysis tools with dynamic instrumentation. *Acm sigplan notices* 40, 6 (2005), 190–200.
- [32] Ramya Jayaram Masti, Devendra Rai, Aanjan Ranganathan, Christian Müller, Lothar Thiele, and Srdjan Capkun. 2015. Thermal Covert Channels on Multi-core Platforms. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, D.C., 865–880. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/masti>
- [33] Ahmad Moghimi, Jan Wichelmann, Thomas Eisenbarth, and Berk Sunar. 2019. Memjam: A false dependency attack against constant-time crypto implementations. *International Journal of Parallel Programming* 47 (2019), 538–570.
- [34] Andrew Moon. [n.d.]. poly1305-donna. <https://github.com/floodyberry/poly1305-donna>.
- [35] Anh Quynh Nguyen and Hoang Vu Dang. [n.d.]. Unicorn: Next Generation CPU Emulator Framework. <https://github.com/unicorn-engine/unicorn>.
- [36] Todd Ouska. 2006. WolfSSL: Embedded TLS Library. <https://www.wolfssl.com/>
- [37] Thomas Pornin. 2016. BearSSL: a smaller SSL/TLS library. <https://bearssl.org/>
- [38] Ashay Rane, Calvin Lin, and Mohit Tiwari. 2015. Raccoon: Closing Digital Side-Channels through Obfuscated Execution. In *24th USENIX Security Symposium, USENIX Security 15, Washington, D.C., USA, August 12-14, 2015*, Jaeyeon Jung and Thorsten Holz (Eds.). USENIX Association, 431–446. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/rane>
- [39] Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. 2017. Dude, is My Code Constant Time?. In *Proceedings of the Conference on Design, Automation & Test in Europe (Lausanne, Switzerland) (DATE ’17)*. European Design and Automation Association, Leuven, BEL, 1701–1706.
- [40] Eyal Ronen, Kenneth G. Paterson, and Adi Shamir. 2018. Pseudo Constant Time Implementations of TLS Are Only Pseudo Secure. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM, 1397–1414. doi:10.1145/3243734.3243775
- [41] Moritz Schneider and Nicolas Dutly. [n.d.]. Breaking Bad Evaluation Infrastructure. <https://github.com/Jumpst3r/breaking-bad-eval-infra>.
- [42] Laurent Simon, David Chisnall, and Ross Anderson. 2018. What You Get is What You C: Controlling Side Effects in Mainstream C Compilers. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–15. doi:10.1109/EuroSP.2018.00009
- [43] The OpenSSL Project. 2003. OpenSSL: The Open Source toolkit for SSL/TLS. (April 2003). www.openssl.org.
- [44] TrustedFirmware. 2009. MbedTLS. <https://www.trustedfirmware.org/projects/mbed-tls/>
- [45] Jo Van Bulck, Frank Piessens, and Raoul Strackx. 2018. Nemesis: Studying Microarchitectural Timing Leaks in Rudimentary CPU Interrupt Logic. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (Toronto, Canada) (CCS ’18)*. Association for Computing Machinery, New York, NY, USA, 178–195. doi:10.1145/3243734.3243822
- [46] Samuel Weiser, David Schrammel, Lukas Bodner, and Raphael Spreitzer. 2020. Big Numbers - Big Troubles: Systematically Analyzing Nonce Leakage in (EC)DSA Implementations. In *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, Srdjan Capkun and Franziska Roesner (Eds.). USENIX Association, 1767–1784. <https://www.usenix.org/conference/usenixsecurity20/presentation/weiser>
- [47] Samuel Weiser, Andreas Zankl, Raphael Spreitzer, Katja Miller, Stefan Mangard, and Georg Sigl. 2018. DATA - Differential Address Trace Analysis: Finding Address-based Side-Channels in Binaries. In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 603–620. <https://www.usenix.org/conference/usenixsecurity18/presentation/weiser>
- [48] Jan Wichelmann, Ahmad Moghimi, Thomas Eisenbarth, and Berk Sunar. 2018. MicroWalk: A Framework for Finding Side Channels in Binaries. In *Proceedings of the 34th Annual Computer Security Applications Conference (San Juan, PR, USA) (ACSAC ’18)*. Association for Computing Machinery, New York, NY, USA, 161–173. doi:10.1145/3274694.3274741
- [49] Yuval Yarom, Daniel Genkin, and Nadia Heninger. 2017. CacheBleed: a timing attack on OpenSSL constant-time RSA. *Journal of Cryptographic Engineering* 7 (2017), 99–112.
- [50] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. 2017. HACL*: A Verified Modern Cryptographic Library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS ’17)*. Association for Computing Machinery, New York, NY, USA, 1789–1806. doi:10.1145/3133956.3134043

A Appendix

In this appendix, we state the data availability of the tools and data we generated in this paper. We then list the initial disclosure statements to the library developers in verbatim (barring some anonymization for the submission). We note that during the disclosure process, the developers sometimes asked for more tests on the patch, different versions, or on even more implementations of theirs. We complied with all of these requests and added these

results to this paper. For instance, the HACL* developers told us about their RSA implementation, which we subsequently tested and added to this paper.

A.1 Data Availability

We will release all research artifacts after publication under an open-source license. This includes the tool we developed to analyze binaries and the software infrastructure to compile a wide variety of crypto libraries with various compilers.

A.2 HACL* Disclosure

We ran a large-scale study of compiler-induced secret-dependent operations on HACL* using dynamic analysis. The study covers 6 processor architectures (x86-64, x86-i386, armv7, aarch64, mips32, RISC-V), two compilers (LLVM and GCC), 17 compiler versions, and 7 optimization levels. HACL* was one of the targets of our study.

Our dynamic analysis framework calls the API of HACL* with different keys and fixes all other randomness. Then we collected a set of traces which we then compared. Any difference in the traces is highlighted as a potential secret-dependent operation. We did not find any issue in most of our experiments of HACL*.

However, we found a code pattern in the secp256 implementation of HACL* that is optimized by some compilers. The resulting binary then contains secret-dependent operations, potentially leading to a side-channel vulnerability.

We understand that you advise people to use formally verified compilers such as CompCert. However, many people use your implementations with off-the-shelf compilers. Therefore, it would be nice if this could be fixed.

We tested the older version of the secp256 implementation (f283af1), and the following analysis and examples are taken from there. We are currently rerunning our analysis for the current master (3e283ef), and we have already found some preliminary secret-dependent operations in the new version. We believe it stems from the same reason as the original code snippet and thus first discuss the old one here.

A.2.1 CMOV. The `cmovznz4` function is supposed to be independent of the input condition `c.in`. However, LLVM versions 13, 14, and 15 compile the snippet to secret-dependent branches (on RISC-V) and secret-dependent memory operations on MIPS, x86-i386, and armv7. An example of this can be seen here: <https://godbolt.org/z/71Wv5b7j>. A table showing the affected compiler versions, architectures, and optimization levels can be seen in Table 3.

A.2.2 Current Master. A similar code pattern to the one shown above is also used in the function `point_mul` in `dist/gcc-compatible/Hacl_P256.c`. Our analysis also indicates that the code path is being executed with some dependency on the secret key. However, this function is vastly more complex and thus harder to pinpoint where the issue comes from. We believe it stems from the same code pattern as above, but we are not 100% confident.

A minimal reproducible example can be found here: <https://godbolt.org/z/6s9daKhr>. The secret-dependent branching instruction that our analysis finds is `jne .LBB0_28` in line 609. A table

arch	toolchain	-O1	-O2	-O3	-Ofast	-Os	-Oz
armv7	LLVM 14	×	×	×	×	✓	✓
	LLVM 15	×	×	×	×	✓	✓
mips32el	LLVM 13	×	×	×	×	✓	✓
	LLVM 14	×	×	×	×	✓	✓
riscv64	LLVM 15	×	×	×	×	✓	✓
	LLVM 13	✓	✓	✓	✓	✓	✓
	LLVM 14	✓	✓	✓	✓	✓	✓
x86-i686	LLVM 15	×	×	×	×	✓	✓
	LLVM 14	×	×	×	×	✓	✓
	LLVM 15	×	×	×	×	✓	✓

Table 3: `cmovnz` in HACL*

arch	toolchain	-O1	-O2	-O3	-Ofast	-Os	-Oz
aarch64	LLVM 13	✓	×	×	×	×	×
	LLVM 14	✓	×	×	×	×	×
	LLVM 15	✓	×	×	×	×	×
armv7	LLVM 13	✓	×	×	×	×	×
	LLVM 14	✓	✓	✓	✓	✓	✓
	LLVM 15	✓	✓	✓	✓	✓	✓
mips32el	LLVM 13	✓	✓	✓	✓	✓	✓
	LLVM 14	✓	✓	✓	✓	✓	✓
	LLVM 15	✓	✓	✓	✓	✓	✓
riscv64	LLVM 13	✓	✓	✓	✓	✓	✓
	LLVM 14	✓	✓	✓	✓	✓	✓
	LLVM 15	✓	✓	✓	✓	×	×
x86-64	LLVM 13	✓	✓	✓	✓	✓	✓
	LLVM 14	✓	✓	✓	✓	✓	✓
	LLVM 15	✓	✓	✓	✓	×	×
x86-i686	LLVM 13	✓	×	×	×	×	×
	LLVM 14	✓	✓	✓	✓	✓	✓
	LLVM 15	✓	✓	✓	✓	✓	✓

Table 4: `cmovnz` in HACL* (current master)

showing the affected compiler versions, architectures, and optimization levels can be seen in Table 4.

A.3 BoringSSL Disclosure

We ran a large-scale study of compiler-induced secret-dependent operations on multiple cryptographic libraries using dynamic analysis. The study covers 6 processor architectures (x86-64, x86-i386, armv7, aarch64, mips32, RISC-V), two compilers (LLVM and GCC), a total of 17 compiler versions, and 7 optimization levels. BoringSSL was one of the targets of our study. We only investigate secret-dependent memory accesses and secret-dependent control flow.

Our dynamic analysis framework uses emulation to execute binaries for different target architectures. We call some of the APIs exposed by BoringSSL in the binaries. To find secret-dependent operations, our tool executes the binaries with different secret keys but fixes all other sources of randomness. We then collect a trace of the executed binaries for each secret key. Any difference in the traces is highlighted as a potential secret-dependent operation. Such secret-dependent operations can lead to an exploitable side-channel

Arch	Compiler	[tp]						
		O0	O1	O2	O3	Ofast	Os	Oz
mips32el	GCC 5.4.0	✓	×	✓	✓	✓	×	×
	GCC 6.3.0	✓	×	✓	✓	✓	×	×
	GCC 7.3.0	✓	×	✓	✓	✓	×	×
	GCC 8.4.0	✓	×	✓	✓	✓	×	×
	GCC 9.3.0	✓	×	✓	✓	✓	×	×
	GCC 10.3.0	✓	×	✓	✓	✓	×	×
	GCC 11.3.0	✓	×	✓	✓	✓	×	×
	LLVM 11	✓	×	×	×	×	×	×
	LLVM 13	✓	×	×	×	×	×	×
	LLVM 14	✓	×	×	×	×	×	×
	LLVM 15	✓	×	×	×	×	×	×

Table 5: `bn_mul_comba8` and `bn_sqr_comba8` in BoringSSL

Arch	Compiler	O0	O1	O2	O3	Ofast	Os	Oz
riscv64	GCC 10.3.0	✓	×	✓	✓	✓	×	×
	GCC 11.3.0	✓	×	✓	✓	✓	×	×

Table 6: `bn_sqr_comba4` in BoringSSL

vulnerability. However, we do not know if the secret-dependent operations that we detect are exploitable at all. Nevertheless, you might be interested in them.

We used the branch `chromium-stable` or `28f96c26` for our analysis.

In general, we did not find any issue that was present in all architectures and all compilers. Such results would indicate issues at the source-code level. However, we found several secret-dependent operations that seem compiler induced as they only appear for some combination of compiler, architecture, and optimization levels. The resulting binary then contains secret-dependent operations, potentially leading to a side-channel vulnerability.

A.3.1 `bn_mul_comba8` and `bn_sqr_comba8`. Our analysis found secret-dependent operations in the function `bn_mul_comba8` and `bn_sqr_comba8` on MIPS (`mips32el`). Both functions are executed during ECDH-P256 and ECDSA. To reproduce it and to demonstrate the secret-dependent operation, we created a small reproducible example using `bn_mul_comba8`: <https://godbolt.org/z/1of34165o>. The resulting binary on the right contains multiple branching instructions that depend on the parameters of this function. One such branch can be found on line 52. Based on the source code, we assume the issue would also pop up in other 32-bit architectures. However, we did not observe that with `x86-i386` and `armv7`. A table showing the affected compiler versions, architectures, and optimization levels can be seen in Table 5.

A.3.2 `bn_sqr_comba4`. We found a similar issue when compiling BoringSSL on RISC-V (`riscv64`) in the function `bn_sqr_comba4`. A small reproducible example can be found in <https://godbolt.org/z/EGdnWhzsz>. Note the branching instruction on line 18.

We observed this behavior with the combinations of compilers, architectures, and optimization levels listed in Table 6.

Arch	Compiler	O0	O1	O2	O3	Ofast	Os	Oz
armv7	LLVM 14	×	✓	✓	✓	✓	✓	✓
	LLVM 15	×	✓	✓	✓	✓	✓	✓
mips32el	LLVM 14	×	✓	✓	✓	✓	✓	✓
	LLVM 15	×	✓	✓	✓	✓	✓	✓
x86-i386	LLVM 14	×	✓	✓	✓	✓	✓	✓
	LLVM 15	×	✓	✓	✓	✓	✓	✓

Table 7: `fiat_p256_point_add` in BoringSSL

A.3.3 `fiat_p256_point_add`. Our analysis flagged a branch in `fiat_p256_point_add` in 32bit architectures (i.e., `armv7`, `x86-i386`, and `mips32el`). However, this function is significantly more complex, which makes it hard to pinpoint the problem. It seems as if the condition `if (is_nontrivial_double)` gets compiled into two distinct branching instructions instead of a single one, and which of these is taken depends on the secret key.

Here is a “short” reproduction of the function: <https://godbolt.org/z/sTGKdbEcK>. We believe LLVM inserts two distinct branches instead of one (line 116 and 120 respectively), and which of these two is taken leaks some information on the input.

Table 7 lists the combinations of compilers, architectures, and optimization levels with which we observed this behavior.

A.4 Botan Disclosure

We ran a large-scale study of compiler-induced secret-dependent operations on multiple cryptographic libraries using dynamic analysis. The study covers 6 processor architectures (`x86-64`, `x86-i386`, `armv7`, `aarch64`, `mips32`, `RISC-V`), two compilers (LLVM and GCC), a total of 17 compiler versions, and 7 optimization levels. Botan was one of the targets of our study. We only investigate secret-dependent memory accesses and secret-dependent control flow.

Our dynamic analysis framework uses emulation to execute binaries for different target architectures. We call some of the APIs exposed by Botan in the binaries. To find secret-dependent operations, our tool executes the binaries with different secret keys but fixes all other sources of randomness. We then collect a trace of the executed binaries for each secret key. Any difference in the traces is highlighted as a potential secret-dependent operation. Such secret-dependent operations can lead to an exploitable side-channel vulnerability. However, we do not know if the secret-dependent operations that we detect are exploitable at all. Nevertheless, you might be interested in them.

We analyzed the release 2.19.3 (15dc32f).

In general, we did not find any issue that was present in all architectures and all compilers. Such results would indicate issues at the source-code level. However, we found several secret-dependent operations that seem compiler induced as they only appear for some combination of compiler, architecture, and optimization levels. The resulting binary then contains secret-dependent operations, potentially leading to a side-channel vulnerability.

A.4.1 `hex_decode`. Our analysis found secret-dependent operations in the function `hex_decode` on recent LLVM versions targeting `riscv64` and `x86-i386`. This function is flagged in various

Arch	Compiler	O0	O1	O2	O3	Ofast	Os	Oz
riscv64	LLVM 14	✗	✓	✓	✓	✓	✓	✓
	LLVM 15	✗	✓	✓	✓	✓	✓	✓
x86-i386	LLVM 14	✗	✓	✓	✓	✓	✓	✓
	LLVM 15	✗	✓	✓	✓	✓	✓	✓

Table 8: hex_decode in Botan

Arch	Compiler	O0	O1	O2	O3	Ofast	Os	Oz
mips32el	GCC 6.4.0	✓	✓	✓	✓	✓	✓	✗
	GCC 7.3.0	✓	✓	✓	✓	✓	✓	✗
	GCC 8.4.0	✓	✓	✓	✓	✓	✓	✗
	GCC 9.3.0	✓	✓	✓	✓	✓	✓	✗
	GCC 10.3.0	✓	✓	✓	✓	✓	✓	✗
	GCC 11.3.0	✓	✓	✓	✓	✓	✓	✗
x86-i386	GCC 7.3.0	✓	✓	✓	✓	✓	✓	✗
	GCC 8.4.0	✓	✓	✓	✓	✓	✗	✗
	GCC 9.3.0	✓	✓	✗	✗	✗	✗	✗
	GCC 10.3.0	✓	✓	✗	✗	✗	✗	✗
	GCC 11.3.0	✓	✓	✗	✓	✓	✗	✗

Table 9: Custom 128-bit type in Botan

cryptographic algorithms: multiple modes of AES, chachapoly1305, curve25519, and HMAC with multiple hash functions. To reproduce it and to demonstrate the secret-dependent operation, we created a small reproducible example: <https://godbolt.org/z/hhs1KWfz7>. The resulting binary on the right contains multiple branching instructions that depend on the parameters of this function. One such branch can be found on line 54.

Table 8 lists the combinations of compilers, architectures, and optimization levels with which we observed this behavior.

A.4.2 Custom 128bit type. Our analysis flags a custom 128bit type and one of its operator as secret-dependent in the implementation of poly1305 and curve25519. More specifically, the `donna128` and its `+=` operator is by some compilers optimized into secret-dependent operations. We replicated the issue and created a minimal reproducible example: <https://godbolt.org/z/P3n3vWsdT>. Note that some compilers only sometimes introduce the branch to skip the carry addition. E.g., in x86-i386, we observed that the compiler did not always produce the same inline binary for the addition operator. For example, <https://godbolt.org/z/xv9fznMfK> is a larger example using x86-i386 in the function `fsquare` where the compiler inserts the branch in line 193.

Table 9 lists the combinations of compilers, architectures, and optimization levels with which we observed this behavior.

A.4.3 key_schedule in GHASH. Our analysis flagged another secret-dependent operation in the function `key_schedule` in GHASH. A reproducible example can be found here: <https://godbolt.org/z/xGh6KjzfH>. Essentially, the compiler understands that the carry variable can only be 1 or 0 and thus checks its value before it performs the XOR.

We found this behavior with LLVM 15 for riscv64 with `-O1`, `-O2`, `-O3`, and `-Ofast`.

A.5 BearSSL Disclosure

We ran a large-scale study of compiler-induced secret-dependent operations on multiple cryptographic libraries using dynamic analysis. The study covers 6 processor architectures (x86-64, x86-i386, armv7, aarch64, mips32, RISC-V), two compilers (LLVM and GCC), a total of 17 compiler versions, and 7 optimization levels. BearSSL was one of the targets of our study. We only investigate secret-dependent memory accesses and secret-dependent control flow.

Our dynamic analysis framework uses emulation to execute binaries for different target architectures. We call some of the APIs exposed by BearSSL in the binaries. To find secret-dependent operations, our tool executes the binaries with different secret keys but fixes all other sources of randomness. We then collect a trace of the executed binaries for each secret key. Any difference in the traces is highlighted as a potential secret-dependent operation. Such secret-dependent operations can lead to an exploitable side-channel vulnerability. However, we do not know if the secret-dependent operations that we detect are exploitable at all. Nevertheless, you might be interested in them.

We used the branch master on 79c060ee for our analysis.

In general, we found one issue that was present in all architectures and all compilers. This indicates that the issue is at the source-code level. In addition, we found several secret-dependent operations that seem compiler induced as they only appear for some combination of compiler, architecture, and optimization level. The resulting binary then contains secret-dependent operations, potentially leading to a side-channel vulnerability.

A.5.1 br_skey_decoder_run. We load the secret keys from a file using `br_skey_decoder_run`, which seems to contain operations that depend on the content of the file. We made sure that our inputs were all the same type of key (i.e., RSA) with different content. We found these issues in every binary, so we assume they are rooted in source-code and not introduced by compiler optimizations. We also did not further evaluate where these issues come from, but you might be interested in them either way.

A.5.2 modpow_opt. The function `modpow_opt` in both `i31` and `i62` were flagged to contain compiler-induced issues on some architectures and compilers. We created small snippets that demonstrate the issue for `i31` and `i62` respectively:

- `i31`: <https://godbolt.org/z/bnE1PrfKG>
- `i62`: <https://godbolt.org/z/nx1crrxv3>

Essentially, the compiler understands the bitmasks computation that is performed using `EQ` and then optimizes/simplifies the constant-time assignment. In `i31` the compiler uses a branch to jump over the assignment and in `i62` the compiler computes the address that is then used for the assignment (the address is then leaked in the actual memory access, i.e., line 10 and 11 in the assembly output of the Godbolt link above).

Table 10 lists the combinations of compilers, architectures, and optimization levels with which we observed this behavior.

A.5.3 br_i31_muladd_small. The compiler once again seems to understand the bitwise arithmetic operations of `MUX`, `EQ`, and `GT` and produces secret-dependent operations in the function

Arch	Compiler	O0	O1	O2	O3	Ofast	Os	Oz
aarch64	LLVM 13	✗	✗	✓	✗	✗	✗	✗
	LLVM 14	✗	✗	✓	✗	✗	✓	✓
	LLVM 14	✗	✓	✗	✗	✗	✓	✓
armv7	LLVM 13	✗	✗	✓	✓	✓	✗	✗
x86-64	LLVM 13	✗	✓	✓	✓	✓	✓	✓
	LLVM 14	✗	✓	✓	✓	✓	✓	✓
	LLVM 15	✗	✓	✓	✓	✓	✓	✓
x86-i386	LLVM 13	✗	✓	✓	✓	✓	✓	✓
	LLVM 14	✗	✓	✓	✓	✓	✓	✓
	LLVM 15	✗	✓	✓	✓	✓	✓	✓

Table 10: modpow_opt in BearSSL

Arch	Compiler	O0	O1	O2	O3	Ofast	Os	Oz
x86-i386	LLVM 13	✗	✓	✓	✓	✓	✗	✗
	LLVM 14	✗	✓	✓	✓	✓	✗	✓
	LLVM 15	✗	✓	✓	✓	✓	✗	✗
riscv64	LLVM 13	✗	✓	✓	✓	✓	✗	✗
	LLVM 14	✗	✓	✓	✓	✓	✗	✓
	LLVM 15	✗	✓	✓	✓	✓	✗	✗

Table 11: br_i31_muladd_small in BearSSL

Arch	Compiler	O0	O1	O2	O3	Ofast	Os	Oz
x86-i386	LLVM 12	✗	✗	✗	✗	✗	✓	✓
	LLVM 13	✗	✓	✓	✓	✓	✓	✓
	LLVM 14	✗	✓	✓	✓	✓	✓	✓
	LLVM 15	✗	✓	✓	✓	✓	✓	✓
riscv64	LLVM 12	✗	✗	✓	✓	✓	✗	✗
	LLVM 13	✗	✓	✓	✓	✓	✗	✗
	LLVM 14	✗	✓	✓	✓	✓	✗	✓
	LLVM 15	✗	✓	✓	✓	✓	✗	✗

Table 12: check_scalar in BearSSL

br_i31_muladd_small. A small reproducible code snippet can be found here: <https://godbolt.org/z/qM9KT31vz>.

Table 11 lists the combinations of compilers, architectures, and optimization levels with which we observed this behavior.

A.5.4 P256 check_scalar. We found a misguided compiler optimization in the function check_scalar in the p256 elliptic curve implementation (both for i62 and i64). A small reproducible example can be found here: <https://godbolt.org/z/zvh6Pjzar>.

Table 12 lists the combinations of compilers, architectures, and optimization levels with which we observed this behavior.